

Online Embedding Compression for Text Classification using Low Rank Matrix Factorization

Anish Acharya ^{*} ^{1,3}, Rahul Goel ¹, Angeliki Metallinou ¹, Inderjit Dhillon ^{2,3}

¹ Amazon Alexa AI, ² Amazon Search Technologies, ³ University of Texas at Austin
achanish, goerahul, ametalli }@amazon.com, isd@a9.com

Abstract

Deep learning models have become state of the art for natural language processing (NLP) tasks, however deploying these models in production system poses significant memory constraints. Existing compression methods are either lossy or introduce significant latency. We propose a compression method that leverages low rank matrix factorization during training, to compress the word embedding layer which represents the size bottleneck for most NLP models. Our models are trained, compressed and then further re-trained on the downstream task to recover accuracy while maintaining the reduced size. Empirically, we show that the proposed method can achieve 90% compression with minimal impact in accuracy for sentence classification tasks, and outperforms alternative methods like fixed-point quantization or offline word embedding compression. We also analyze the inference time and storage space for our method through FLOP calculations, showing that we can compress DNN models by a configurable ratio and regain accuracy loss without introducing additional latency compared to fixed point quantization. Finally, we introduce a novel learning rate schedule, the Cyclically Annealed Learning Rate (CALR), which we empirically demonstrate to outperform other popular adaptive learning rate algorithms on a sentence classification benchmark.

1 Introduction

Deep learning has achieved great success in various NLP tasks such as sequence tagging (?), (?) and sentence classification (?; ?). While traditional machine learning approaches extract hand-designed and task-specific features, which are fed into a shallow model, neural models pass the input through several feedforward, recurrent or convolutional layers of feature extraction that are trained to learn automatic feature representations. However, deep neural models often have large memory-footprint and this poses significant deployment challenges for real-time systems that typically have memory and computing power constraints. For example, mobile devices tend to be limited by their CPU speed, memory and battery life, which poses significant size constraints for models embedded on such devices. Similarly, models deployed on servers need to serve millions of re-

quests per day, therefore compressing them would result in memory and inference cost savings.

For NLP specific tasks, the word embedding matrix often accounts for most of the network size. The embedding matrix is typically initialized with pretrained word embeddings like Word2Vec, (?) FastText (?) or Glove (?) and then fine-tuned on the downstream tasks, including tagging, classification and others. Typically, word embedding vectors are 300-dimensional and vocabulary sizes for practical applications could be up to 1 million tokens. This corresponds to up to 2GBs of memory and could be prohibitively expensive depending on the application. For example, to represent a vocabulary of 100K words using 300 dimensional Glove embeddings the embedding matrix would have to hold 60M parameters. Even for a simple sentiment analysis model the embedding parameters are 98.8% of the total network (?).

In this work, we address neural model compression in the context of text classification by applying low rank matrix factorization on the word embedding layer and then re-training the model in an online fashion. There has been relatively less work in compressing word embedding matrices of deepNLP models. Most of the prior work compress the embedding matrix offline outside the training loop either through hashing or quantization based approaches (?; ?; ?). Our approach includes starting from a model initialized with large embedding space, performing a low rank projection of the embedding layer using Singular Value Decomposition (SVD) and continuing training to regain any lost accuracy. This enables us to compress a deep NLP model by an arbitrary compression fraction (p), which can be pre-decided based on the downstream application constraints and accuracy-memory trade-offs. Standard quantization techniques do not offer this flexibility, as they typically allow compression fractions of $1/2$ or $1/4$ (corresponding to 16-bit or 8-bit quantization from 32-bits typically).

We evaluate our method on text classification tasks, both on the benchmark SST2 dataset and on a proprietary dataset from a commercial artificial agent. We show that our method significantly reduces the model size (up to 90%) with minimal accuracy impact (less than 2% relative), and outperforms popular compression techniques including quantization (?) and offline word embedding size reduction (?).

A second contribution of this paper is the introduction of a novel learning rate schedule, we call Cyclically An-

^{*}Corresponding Author

nealed Learning Rate (CALR), which extends previous works on cyclic learning rate (?) and random hyperparameter search (?). Our experiments on SST2 demonstrate that for both DAN (?) and LSTM (?) models CALR outperforms the current state-of-the-art results that are typically trained using popular adaptive learning like AdaGrad.

Overall, our contributions are:

- We propose a compression method for deep NLP models that reduces the memory footprint through low rank matrix factorization of the embedding layer and regains accuracy through further finetuning.
- We empirically show that our method outperforms popular baselines like fixed-point quantization and offline embedding compression for sentence classification.
- We provide an analysis of inference time for our method, showing that we can compress models by an arbitrary configurable ratio, without introducing additional latency compared to quantization methods.
- We introduce CALR, a novel learning rate scheduling algorithm for gradient descent based optimization and show that it outperforms other popular adaptive learning rate algorithms on sentence classification.

2 Related Work

In recent literature, training overcomplete representations is often advocated as overcomplete bases can transform local minima into saddle points (?) or help discover robust solutions (?). This indicates that significant model compression is often possible without sacrificing accuracy and that the model’s accuracy does not rely on precise weight values (?). Most of the recent work on model compression exploits this inherent sparsity of overcomplete representations. These works include low precision computations (?; ?) and quantization of model weights (?; ?). There are also methods which prune the network by dropping connections with low weights (?; ?) or use sparse encoding (?). These methods in practice often suffer from quantization loss especially if the network is deep due to a large number of low-precision multiplications during forward pass. Quantization loss is hard to recover through further training due to the non-trivial nature of backpropagation in low precision (?). There has been some work on compression aware training (?) via model distillation (?) that addresses this gap in accuracy. However, both quantized distillation and differentiable quantization methods introduce additional latency due to their slow training process that requires careful tuning, and may not be a good fit for practical systems where a compressed model needs to be rapidly tuned and deployed in production.

Low-rank matrix factorization (LMF) is a very old dimensionality reduction technique widely used in the matrix completion literature. For example, see (?) and references therein. However, there has been relatively limited work on applying LMF to deep neural models. (?) used low rank matrix factorization for neural speech models. While (?) reduces parameters of DNN before training, (?) restructures the network using SVD introducing bottleneck layers in the weight matrices. However, for typical NLP tasks, introducing bottleneck layers between the deep layers of the model

does not significantly decrease model size since the large majority of the parameters in NLP models come from the input word embedding matrix. Building upon this prior work, we apply LMF based ideas to reduce the embedding space of NLP models. Instead of exploiting the sparsity of parameter space we argue that for overcomplete representation the input embedding matrix is not a unique combination of basis vectors (?). Thus, it is possible to find a lower dimensional parameter space which can represent the model inputs with minimal information loss. Based on this intuition, we apply LMF techniques for compressing the word embedding matrix which is the size bottleneck for many NLP tasks.

3 Methodology

3.1 Low-rank Matrix Factorization (LMF) for Compressing Neural Models

Low-rank Matrix Factorization (LMF) exploits latent structure in the data to obtain a compressed representation of a matrix. It does so by factorization of the original matrix into low-rank matrices. For a full rank matrix $W \in \mathbb{R}^{m \times n}$ of rank r , there always exists a factorization $W = W_a \times W_b$ where $W_a \in \mathbb{R}^{m \times r}$ and $W_b \in \mathbb{R}^{r \times n}$. Singular Value Decomposition (SVD) (?) achieves this factorization as follows:

$$W_{m \times n} = U_{m \times m} \Sigma_{m \times n} V_{n \times n}^t \quad (1)$$

where $\Sigma_{m \times n}$ is a diagonal rectangular matrix of singular values of descending magnitude. We can compress the matrix by choosing the k largest singular values, with $k < m$ and $k < n$. For low rank matrices, the discarded singular values will be zero or small, therefore the following will approximately hold:

$$W_{m \times n} \approx U_{m \times k} \Sigma_{k \times k} V_{k \times n}^t \quad (2)$$

Therefore, the original matrix W is compressed into two matrices:

$$W_a = U \in \mathbb{R}^{m \times k} \quad (3)$$

$$W_b = \Sigma_{k \times k} V_{k \times n}^t \in \mathbb{R}^{k \times n} \quad (4)$$

The number of parameters are reduced from $m \times n$ to $k \times (m + n)$. Therefore, to achieve a p fraction parameter reduction, k should be selected as:

$$(p \times m \times n) = k \times (m + n) \\ k = \left\lfloor \frac{pmn}{m+n} \right\rfloor; k \in \mathbb{Z}^+ \quad (5)$$

The value k can be varied depending on the desired compression fraction p as long as $k \geq 1$. In practice, being able to decide an arbitrary compression fraction p is an attractive property for model compression, since it allows choosing compression rates based on the accuracy-memory trade-offs of a downstream application. The low rank matrix factorization operation is illustrated in Figure 1, where a single neural network matrix (layer) is replaced by two low rank matrices (layers).

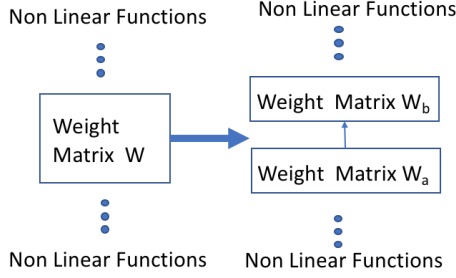


Figure 1: Replacing one neural network matrix with two low rank matrices

Compressing the Word Embedding Matrix For NLP tasks, typically the first layer of the neural model consists of an embedding lookup layer that maps the words into real-valued vectors for processing by subsequent layers. The words are indices i of a word dictionary of size D while the word embeddings are vectors of size d . This corresponds to a lookup table $E(i) = W_i$ where $W \in \mathbb{R}^{d \times |D|}$. The word embeddings are often trained offline on a much larger corpus, using algorithms like Word2Vec, GloVe etc, and are then fine-tuned during training on the downstream NLP task. Our method decomposes the embedding layer in an online fashion during training using eq.(5). W_a becomes the new embedding layer and W_b becomes the next layer. Continuing backpropagation over this new network structure fine-tunes the low rank embedding space and regains any accuracy loss within a few epochs.

3.2 Cyclically Annealed Learning Rate

Due to the non-convex nature of the optimization surface of DNNs, gradient based algorithms like stochastic gradient descent (SGD) are prone to getting trapped in suboptimal local minima or saddle points. Adaptive learning rates like Adam (?) and Adagrad (?) try to solve this problem by tuning learning rate based on the magnitude of gradients. While in most cases they find a reasonably good solution, they usually can't explore the entire gradient landscape. (??) indicates periodic random initialization or warm restarts often helps in better exploration of the landscape while (?) showed that letting the learning rate(LR) vary cyclically (Cyclic learning rate(CLR)) helps in faster convergence. In this work we further refine CLR and propose a simulated annealing inspired LR update policy we call Cyclically Annealed Learning Rate (CALR). CALR is described in algorithm (1) and Fig. 2b. In addition to varying LR in a CLR style triangular windows(Fig. 2a), CALR exponentially decays the upper bound LR_{UB} of the triangular window from its initial value. When the learning rate decreases to a lower bound LR_{LB} , we increase the upper bound to its initial value LR_{UB} and continue with the LR updates. This exponential decay ensures slow steps on the optimization landscape. Intuitively, the proposed warm restarts, motivated by simulated annealing, make CALR more likely to escape from local minima or saddle points and enables better exploration of the parameter space. CALR should have similar

but less aggressive effect as random initialization or warm restarts of LR. While exponentially decayed small cyclic windows let CALR carefully explore points local to the current solution, cyclic temperature increase helps it jump out and explore regions around other nearby local minima. We verified this intuition empirically for our classification experiments (Table 1, Section 5.4), where CALR was able to achieve further improvement compared to CLR and Ada-grad.

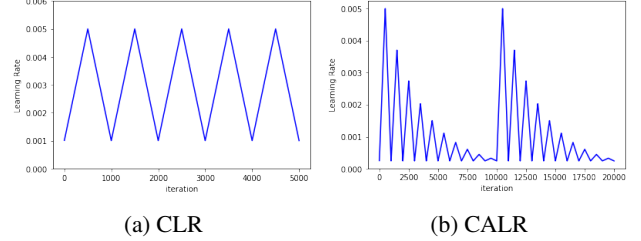


Figure 2: Comparison of CLR and CALR update Policy

Algorithm 1 Cyclically Annealed Learning Rate Schedule

```

1: procedure CALR(Iteration, Step Size,  $LR_{LB}$ ,  $LR_{UB}$ )
2:    $LR_{UB} \leftarrow LR_{UB}(init)$ 
3:    $LR_{LB} \leftarrow LR_{LB}(init)$ 
4:   for each EPOCH do
5:      $LR_{UB} \leftarrow LR_{UB} \times \exp(Decay)$ 
6:     if  $LR_{UB} \leq LR_{LB}$  then
7:        $LR_{UB} \leftarrow LR_{UB}(init)$ 
8:     for each Training BATCH do
9:        $LR \leftarrow CLR(Iteration, StepSize, LR_{LB}, LR_{UB})$ 
10:    return  $LR$ 

1: procedure CLR(Iteration, Step Size,  $LR_{LB}$ ,  $LR_{UB}$ )
2:    $\triangleright$  this procedure implements a triangular window of
   width StepSize and height  $LR_{UB} - LR_{LB}$ 
3:    $Bump \leftarrow \frac{LR_{UB} - LR_{LB}}{StepSize}$ 
4:    $Cycle \leftarrow (Iteration) \bmod (2 \times StepSize)$ 
5:   if  $Cycle < StepSize$  then
6:      $LR \leftarrow LR_{LB} + (Cycle \times Bump)$ 
7:   else
8:      $LR \leftarrow LR_{UB} - (Cycle - StepSize) \times Bump$ 
9:   return  $LR$ 

```

3.3 Baselines

We compare our proposed method with two commonly baselines for neural model compression.

Fixed Point Weight Quantization (Baseline 1) We compare our approach with widely used post-training fixed point quantization (??) method where the weights are cast into a lower precision. This requires less memory to store them and doing low precision multiplications during forward propagation improves inference latency.

Offline Embedding Compression (Baseline 2) In this set of baseline experiments we compress the embedding matrix

in a similar way using SVD but we do it as a preprocessing step similar to (?; ?). For example, if we have a 300 dimensional embedding and we want a 90% parameter reduction we project it onto a 30 dimensional embedding space and use this low dimensional embedding to train the NLP model.

4 Analysis

In this section we analyze space and latency trade-off in compressing deep models. We compare the inference time and storage space of floating point operations(FLOPs) between our method and Baseline1(sec.3.3) on a 1-layer Neural Network(NN) whose forward pass on input X can be represented as:

$$f(X) = \sigma(XW) \quad (6)$$

where $X \in \mathbb{R}^{1 \times m}$ and $W \in \mathbb{R}^{m \times n}$ and full rank. Our method reduces W into two low rank matrices W_a and W_b converting (6) into two layer NN represented as:

$$f(X) = \sigma((X \times W_a) \times W_b) \quad (7)$$

where $W_a \in \mathbb{R}^{m \times k}$ and $W_b \in \mathbb{R}^{k \times n}$ are constructed as described in eq.(3) and eq.(4) choosing k as in eq.(5). Whereas, Baseline1 casts the elements of W to lower precision without restructuring the network. Thus, the quantized representation of a 1-layer NN is same as eq.(6) where W_i is cast to a lower bit precision and eq. (7) is the corresponding representation in our approach where W_i remains in its original precision.

4.1 Space Complexity Analysis

On Baseline1 each weight variable uses less number of bits for storage. Quantizing the network weights into low precision, we achieve space reduction by a fraction $\frac{B_Q}{B_S}$ where B_S bits are required to store a full precision weight and B_Q bits are needed to store low precision weight. In contrast, our approach described in section 3.1, achieves more space reduction as long as we choose k (from eq (5)) such that:

$$p < \frac{B_Q}{B_S} \quad (8)$$

4.2 Time Complexity Analysis

Total number of FLOPs in multiplying two matrices (?) is given by: $FLOP(A \times B) = (2b - 1)ac \sim O(abc)$ where $A \in \mathbb{R}^{a \times b}$, $B \in \mathbb{R}^{b \times c}$ and $AB \in \mathbb{R}^{a \times c}$. Assuming one FLOP in a low precision matrix operations takes T_Q time and on a full precision matrix it takes T_S time, for the model structure given by eq. (6), we have: $a = 1, b = m, c = n$. Hence, for the forward pass on a single data vector $X \in \mathbb{R}^{1 \times m}$ a quantized model uses F_Q flops where:

$$F_Q = (2m - 1)n \quad (9)$$

Our algorithm restructures the DNN given in eq. (6) into eq. (7) yielding $F_S = F_{S1} + F_{S2}$ flops. where:

$$\begin{aligned} F_{S1} &= (2m - 1)k, F_{S2} = (2k - 1)n \\ F_S &= 2(m + n)k - (n + k) \end{aligned} \quad (10)$$

The number of FLOPs in one forward pass our method would be less than that of in Baseline1 if $F_Q < F_S$ or equivalently:

$$\begin{aligned} (2m - 1)n &> 2(m + n)k - (n + k) \\ k &< \frac{2mn}{2(m + n) - 1} \end{aligned} \quad (11)$$

Under the reasonable assumption that $2(m + n) \gg 1$ or equivalently $2(m + n) - 1 \approx 2(m + n)$, eq. (11) becomes:

$$k < \frac{mn}{(m + n)} \quad (12)$$

Eq. (12) will hold as long as k in our method is chosen according to eq.(5) (since the fraction p is by definition $p < 1$) However, guaranteeing less FLOPs does not guarantee faster inference since quantized model requires less time per FLOP. Our method guarantees faster inference if the following condition holds:

$$\begin{aligned} F_Q T_Q &> F_S T_S \\ (2m - 1)n T_Q &> [(2m - 1)k + (2k - 1)n] T_S \\ \frac{(2m - 1)n}{[(2m - 1)k + (2k - 1)n]} &> \frac{T_S}{T_Q} \end{aligned} \quad (13)$$

where T_S and T_Q are time taken per FLOP for our method and Baseline1 respectively. Again, we can make the reasonable assumptions that $2m \gg 1$ and $2k \gg 1$, therefore $2m - 1 \approx 2m$ etc. Then eq. (13) becomes:

$$\frac{mn}{(m + n)k} > \frac{T_S}{T_Q} \quad (14)$$

Plugging k from eq.(5) we have:

$$p < \frac{T_Q}{T_S} \quad (15)$$

If we assume that the time per FLOP is proportional to the number of bits B for storing a weight variable, e.g., $T_Q \sim B_Q$ and $T_S \sim B_S$ then selecting p such that $p < \frac{B_Q}{B_S}$ will ensure that eq. (15) holds. In most practical scenarios time saving is sublinear in space saving which means eq.(8) is necessary and sufficient to ensure our method has faster inference time than fixed point quantization(Baseline1).

5 Experiments and Results

Our experiments are focused on sentence classification task.

5.1 Datasets:

For all the experiments reported we have used the following two sentence classification datasets:

Stanford Sentiment Treebank (SST2) SST2 (?) is a common text classification benchmark dataset that contains movie reviews, to be classified according to their sentiment. On an average the reviews are about 200 words long. The dataset contains standard train, dev, test splits and binary sentiment labels.

Books Intent Classification Dataset We use a proprietary

dataset of around 60k annotated utterances of users interacting with a popular digital assistant about books related functionality. Each utterance is manually labeled with the user intent e.g., search for a book, read a book and others, with the task being to predict the user intent from a set of 20 intents. We choose 50k utterances as our training data, 5k as test data and 5k as dev data.

5.2 Models:

We evaluate our compression strategy on two types of widely used neural models:

Long Short Term Memory(LSTM) LSTMs, are powerful and popular sequential models for classification tasks. The LSTM units are recurrent in nature, designed to handle long term dependencies through the use of input, output and forget gates and a memory cell. For input $X = x_1, \dots, x_T$ and corresponding word embedding input vectors $e_i, i = 1, \dots, T$, LSTM computes a representation r_t at each word t , which is denoted as $r_t = \phi(e_t, r_{t-1})$. Here, for sentence classification, we used the representation obtained from the final timestep of the LSTM and pass it through a softmax for classification: $r^{sent} = r_T^{forw}$, and $\hat{S} = \text{softmax}(W_s r^{sent} + b_s)$ where r^{sent} is the sentence representation, and $\hat{S}$ is the label predicted. In our experiments with LSTM, we used 300-dimensional Glove embeddings to initialize the embedding layer for all the methods, e.g., baselines 1, 2 and our proposed compression method. Our vocabulary size V contains only the words that appear in the training data, and we compress the input word embedding matrix W (original size $V \times 300$).

Deep Averaging Network (DAN) DAN is a bag-of-words neural model that averages the word embeddings in each input utterance to create a sentence representation r^{sent} that is passed through a series of fully connected layers, and fed into a softmax layer for classification. Assume an input sentence X of length T and corresponding word embeddings e_i , then the sentence representation is: $r^{sent} = \frac{1}{T} \sum_{i=1}^T e_i$. In our setup, we have two fully connected layers after the sentence representation r^{sent} , of sizes 1024 and 512 respectively. For the DAN experiments we used the DAN-RAND (?) variant where we take a randomly initialized 300 dimensional embedding. To make the comparison fair, all experiments with DAN models, including quantization (baseline 1), the offline compression method (baseline 2) and the proposed compression method are done using randomly initialized word embedding matrices. For example, our baseline 2 is to use appropriate low-rank randomly initialized embedding matrix.

5.3 Experimental Setup

Our experiments are performed on 1 Tesla K80 GPU, using SGD optimizer with CALR policy (Sec.3.2), initial learning rate upper bound of 0.001. We use dropout of 0.4 between layers and L2 regularization with weight of 0.005. Our metrics include accuracy and model size. Accuracy is simply the fraction of utterances with correctly predicted label. The model size is calculated as sum of (.data + .index + .meta) files stored by tensorflow.

5.4 Results on Uncompressed Models

To effectively train our uncompressed benchmark networks we experimented with different adaptive learning rate schedules. Table 1 empirically demonstrates the effectiveness of our proposed learning rate update policy (CALR) on the SST2 test set. For both the models CALR outperforms the corresponding state-of-the-art sentence classification accuracy reported in the literature. For LSTM we improve the accuracy from 84.9% (?) to 86.03% (1.33% relative improvement) whereas on DAN-RAND we are able to improve from 83.2% (?) to 84.61% (1.7% relative improvement). We also report results on training the network with CLR without our cyclic annealing. For both DAN-RAND and LSTM, CLR achieves performance that is similar to the previously reported state-of-the-art. The proposed CALR outperforms CLR, which supports the effectiveness of the proposed CALR policy.

5.5 Results on SST2

Table 2 shows compression and accuracy results on the SST2 test dataset for our proposed methods, the two baselines described in Section 3.3, and for two types of models: LSTM (upper table rows) and DAN-RAND (lower table rows). For both types of models, we first report the original uncompressed model size and accuracy. For the quantization baseline (baseline 1), we report size and accuracy numbers for 8-bit and 16-bit quantization. For the SVD-based methods, both for the offline embedding compression (baseline 2) and our proposed compression method, we compare size and accuracy for different percentages of model size reduction R , where $R = 1 - p$. This reduction corresponds to different fractions p and to different number of selected singular values k (Section 3.1).

From Table 2, we observe that our proposed compression method outperforms both baselines for both types of models. For LSTM, our method achieves $R = 90\%$ compression with 85.11 accuracy e.g., only 1% rel. degradation compared to the uncompressed model, while the offline embedding compression (baseline 2) achieves 82.54 (4% rel degradation vs uncompressed). Looking at $R = 50\%$ our method achieves accuracy of 85.67 (0.4% rel degradation) vs 84.24 for baseline 2 (2% rel degradation) and 85.08 for baseline 1 which does 16-bit compression (1% rel degradation). For DAN-RAND and for $R = 90\%$ compression, our method achieves accuracy of 83.11 which corresponds to 1.7% rel

Model	LR Schedule	Acc.
Uncompressed LSTM Model		
?	AdaGrad	84.90
?	CLR	84.71
Ours	CALR	86.03
Uncompressed DAN-RAND Model		
?	AdaGrad	83.20
?	CLR	83.84
Ours	CALR	84.61

Table 1: Comparison of initial Uncompressed Model for different Learning Rate Schedules on SST2

Model	R(%)	Size(MB)	Acc.	Acc.
Uncompressed LSTM model				
LSTM	0	53.25	86.03	-
Quantized model (Baseline 1)				
16 bit	50	30.16	85.08	-
8 bit	75	18.21	85.01	-
SVD: Proposed vs Offline Compression (Baseline 2)				
			Proposed	Baseline2
LSTM	10	48.66	85.72	85.45
	30	38.52	85.68	84.95
	50	28.45	85.67	84.24
	70	18.38	85.45	83.09
	90	6.94	85.11	82.54
Uncompressed DAN model				
DAN	0	52.84	84.61	-
Quantized model (Baseline 1)				
16 bit	50	28.36	83.18	-
8 bit	75	18.13	82.94	-
SVD: Proposed vs Offline Compression (Baseline 2)				
			Proposed	Baseline2
DAN	10	47.29	84.24	83.47
	30	37.23	83.83	83.31
	50	27.16	83.72	82.87
	70	17.18	83.67	82.86
	90	6.21	83.11	82.59

Table 2: Compression and accuracy results on SST2 dataset. R(%) refers to percentage of model size reduction, Size is the model size, and Acc is the classification accuracy. All DAN models use the DAN-RAND variant.

degradation vs an uncompressed model, and outperforms baseline 2 that has accuracy of 82.59 (2.4% rel. degradation). Similar trends are seen across various size reduction percentages R .

Comparing LSTM vs DAN-RAND models in terms of the gain we achieve over quantization, we observe that for LSTM our compression outperforms quantization by a large margin for the same compression rates, while for DAN-RAND the gain is less pronounced. We hypothesize that this is because LSTMs have recurrent connections and a larger number of low precision multiplications compared to DAN, which has been shown to lead to worse performance(?). Thus, we expect our method to benefit the most over quantization when the network is deep.

To illustrate how our proposed method benefits from re-training the model after compressing the embedding input matrix, in Fig 3 we plot the dev set accuracy for SST2 across training mini-batches for different compression percentages R and for both LSTM and DAN-RAND. For comparison, we plot the dev accuracy of 8-bit and 16-bit quantization (baseline 1) that remains stable across epochs as the quantized model is not retrained after compression. As a further comparison, we plot the uncompressed model accuracy on the dev which is kept stable as well (we assume this as our benchmark point). We observe that, while our compressed model starts off at a lower accuracy compared to both the uncompressed model and the quantization baseline, it rapidly improves with training and stabilizes at an accuracy that is higher than the quantization method and comparable to the

Model	R(%)	Size(MB)	Acc.	Acc.
Uncompressed LSTM model				
LSTM	0	29.34	91.78	-
Quantized model (Baseline 1)				
16 bit	50	17.92	90.16	-
8 bit	75	10.05	89.96	-
SVD: Proposed vs Offline Compression (Baseline 2)				
			Proposed	Baseline2
LSTM	10	26.76	91.71	91.07
	30	21.69	91.63	90.97
	50	16.62	91.54	90.87
	70	10.31	91.47	90.58
	90	4.89	90.94	89.83
Uncompressed DAN model				
DAN	0	37.03	90.18	-
Quantized model (Baseline 1)				
16 bit	50	25.68	89.38	-
8 bit	75	16.41	88.86	-
SVD: Proposed vs Offline Compression (Baseline 2)				
			Proposed	Baseline2
DAN	10	33.96	90.15	87.35
	30	29.43	90.12	87.34
	50	23.16	89.47	87.14
	70	18.21	89.55	86.15
	90	13.08	89.23	83.72

Table 3: Compression and accuracy results on the Books Intent dataset. R(%) refers to percentage of model size reduction, Size is the model size, and Acc is the classification accuracy. All DAN models use the DAN-RAND variant.

original uncompressed model. This indicates that re-training after SVD compression enables us to fine-tune the layer parameters W_a and W_b towards the target task, and allows us to recover most of the accuracy.

5.6 Results on Books Intent Dataset

Table 3 shows compression and accuracy results on the proprietary Books Intent test set for our proposed methods vs the two baselines, for the LSTM and DAN-RAND models. Overall, we make similar observations as for the SST2 dataset. Our proposed method achieves better accuracy across compression percentages compared to both offline embedding compression and quantization for both LSTM and DAN-RAND. For example, for LSTM and $R=90\%$ compression we achieve accuracy of 90.94% which corresponds to 1% degradation compared to the uncompressed model, while offline embedding compression (baseline 2) achieves accuracy of 89.83 (2% degradation). We also examined the plots of dev set accuracy while re-training the proposed compressed model across mini-batches and observed similar trends as for SST2 (plots are omitted for brevity).

5.7 Results on Inference Time

In Table 4, for both LSTM and DAN-RAND, we report inference times for the SST2 test set for different compression fractions using the proposed compression vs inference times for 16-bit and 8-bit fixed point quantization (baseline 2). As expected, for our method we see a decrease in inference time as the compression fraction (R) increases. We observe similar trends for the quantization method, where 8-bit is faster

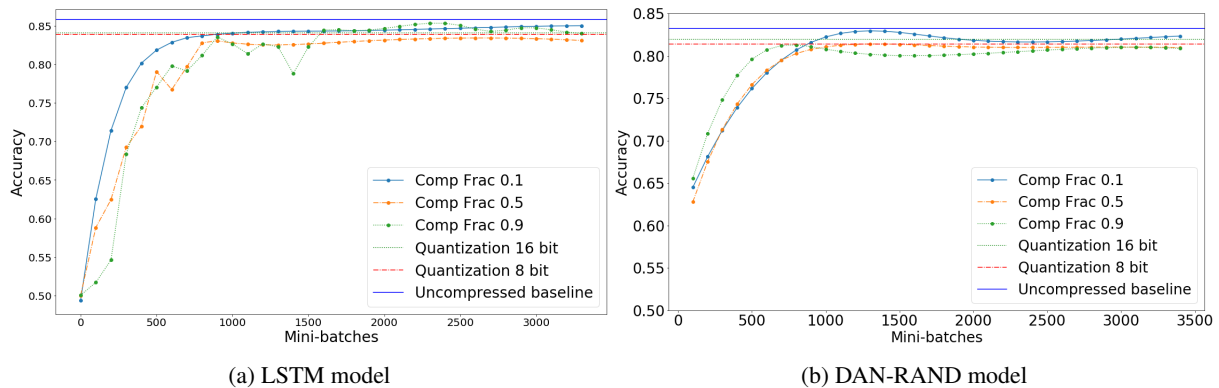


Figure 3: Dev set accuracy vs training mini batches for various compression percentages R for the SST2 dataset

	LSTM		DAN-RAND	
R(%)	Proposed	Baseline 2	Proposed	Baseline 2
10	10.71	-	1.38	-
50	9.28	9.23	0.89	0.93
70	9.21	-	0.62	-
75	9.11	9.08	0.59	0.62
90	8.46	-	0.45	-

Table 4: Inference Time on the SST2 test set (seconds)

than 16-bit. For similar compression rates (16-bit equivalent to 50% compression and 8-bit is equivalent to 75% compression) our method and baseline 2 (fixed-point-quantization) show similar inference times. Therefore, our method does not introduce any significant latency during inference while regaining the accuracy.

6 Conclusions and Future Work

In this work, we have proposed a neural model compression method based on low rank matrix factorization that can reduce the model memory footprint by an arbitrary proportion, that can be decided based on accuracy-memory trade-offs. Our method consists of compressing the model and then re-training it to recover accuracy while maintaining the reduced size. We have evaluated this approach on text classification tasks and showed that we can achieve up to 90% model size compression for both LSTM and DAN models, with minimal accuracy degradation (1%-2% relative) compared to an uncompressed model. We also showed that our method empirically outperforms common model compression baselines such as fixed point quantization and offline word embedding compression for the classification problems we examined. We have also provided an analysis of our method’s effect on the model inference time and showed under which conditions it can achieve faster inference compared to model quantization techniques. An additional contribution of this work is the introduction of a novel learning rate scheduling algorithm, the Cyclically Annealed Learning Rate (CALR). We compared CALR to other popular adaptive learning rate algorithms and showed that it leads to better performance on the SST2 benchmark dataset. In future, we plan to evaluate

the proposed compression method and the proposed CALR schedule on a larger variety of NLP tasks including sequence tagging and sequence to sequence modeling.

Laboriosam autem corrupti perspicatis laborum, voluptatem facilis voluptatibus unde officia nihil, accusantium doloribus adipisci quibusdam soluta assumenda porro dolores molestias, suscipit quibusdam corrupti quo repudiandae consequuntur enim, cumque libero officia soluta?Reprehenderit ducimus voluptates eos doloribus aliquam, atque consequatur assumenda iste reprehenderit sunt culpa incidunt, deserunt sapiente corrupti minus suscipit doloremque, impedit quibusdam voluptatum minus nulla nihil est accusantium ipsum illum corporis, delectus recusandae illo iste accusamus quas eum obcaecati nemo deleniti?Cum repellendus eligendi asperiores aliquam rem animi reiciendis delectus, quaerat labore quis cumque hic beatae non amet minima, ab facilis earum illo est error, quis architecto est ipsam itaque dolorem?Fuga optio quis expedita, temporibus veniam architecto eum ex vero a doloremque id, velit illo molestias atque recusandae doloribus deserunt sit nulla labore possimus, non ut eveniet placeat consectetur quas praesentium cum recusandae nobis, fugit perspicatis ab sint architecto unde officia iste.Iusto assumenda fuga voluptatum neque voluptatibus numquam culpa quod dolor incidunt, fugiat molestiae esse non earum delectus.Illum similique voluptate id iste, sequi molestiae quas quia ab dolorem, error reprehenderit obcaecati praesentium aliquam dicta numquam?Voluptas sed quae libero vitae quis voluptate pariatur perspicatis culpa dicta, officia facere ab accusantium eveniet minus qui aliquid temporibus itaque, maiores quas est repellendus deleniti odit suscipit atque ex, totam dolores ad explicabo repudiandae possimus aliquam est quis, provident excepturi eum ad.Aut eum sapiente nihil asperiores explicabo sit vel id ab corporis quo, rem libero enim, quod voluptates aliquid cupiditate?Modi deleniti commodi, praesentium molestiae dolor eum doloribus labore repellat autem amet dolore expedita.Officia dolorum neque accusamus porro sequi, excepturi adipisci tempora iure sapiente eligendi illum eius cupiditate nostrum, ipsa necessitatibus deserunt vel amet maxime a, molestias autem voluptas?Numquam nemo saepe expedita pariatur labore consequuntur fugiat reiciendis harum, itaque a quaerat esse delectus corrupti natus accusamus in-

ventore sapiente, optio itaque deleniti, ab sed ad quos ipsam. Numquam iure et asperiores eveniet omnis, ipsa recusandae quidem mollitia optio culpa veritatis, animi corporis